Symmetric Stigmergy with Foundation-Model Agents: Pheromone-Field Coordination for Decentralized Artifact Refinement

Rolando Rene Rodriguez, Jr.[1*]

[1]Independent Researcher, Corpus Christi, TX, United States.

Corresponding author(s). E-mail(s): rrrodzilla@proton.me;

Abstract

Stigmergy, coordination through marks left in a shared environment, has mostly been applied with simple reactive agents. We instantiate Parunak's symmetric-stigmergic schema in a foundation-model multi-agent system, where reasoning agents coordinate entirely through a shared artifact: each observes a region's local quality signals (the sensed pressure field) and proposes a patch, with no planner or manager making content decisions and no inter-agent messages. The instantiation realizes each primitive of the schema, including the environment-to-agent arrow that classic asymmetric stigmergy lacks: when the field stops improving, the environment escalates agent sampling and model capability. Two pheromone stores evaporate each tick so that stale patches are forgotten, and a one-hop propagation operator spreads activation along the temporal structure of the domain. Foundation models and stigmergy each supply what the other lacks: foundation models propose the open-ended actions that classic stigmergy could not enumerate, and the symmetric schema supplies the objective, pressure-based criterion for combining those proposals. On meeting room scheduling across a 1350-trial grid, we characterize where this decentralized coordination outperforms hierarchical control and where it does not. On medium and hard problems it solves **37%** and **17%** of instances while the centralized baselines solve at most one in ninety; on the smallest instances the gap narrows to a finite multiple. We give one convergence result: aligned local descent terminates in time bounded by initial pressure.

Keywords: stigmergy, symmetric stigmergy, multi-agent systems, LLM agents, emergent coordination

1 Introduction

We instantiate the symmetric-stigmergic schema of Parunak (2025) in a foundation-model multi-agent system, and ask where coordination through a shared artifact can replace hierarchical control. Foundation-model multi-agent systems mostly coordinate through explicit dialogue: planners decompose tasks, managers delegate, and workers execute under hierarchical supervision Wu et al. (2023); Hong et al. (2024); Li et al. (2023). That overhead scales poorly, and a growing body of large-scale studies documents its costs Kim et al. (2025); Zhang et al. (2025); Qian et al. (2025).

A second strand suggests these systems coordinate better through the environment than through messages. Language-model agents perform best when coordination rests on environmental state and worst when it requires reasoning about other agents' minds Agashe et al. (2025), and generative-agent societies organize joint activity through shared co-presence rather than broadcast Park et al. (2023). This is stigmergy, coordination through marks left in a shared environment Grassé (1959). Classic stigmergy is asymmetric, the agent changing the environment but not the reverse; the recent symmetric extension adds the missing arrow, letting the environment also act on agent state, which is exactly what an adaptive multi-agent system built on heterogeneous models needs Parunak (2025).

Coordination operates on a shared *artifact*, a structured object divided into *regions*; in our experiments the artifact is a week's meeting schedule and the regions are its time blocks. Each region's quality deficits are aggregated into a single scalar, its *pressure*, and the *pressure field* is that value over all regions. We reserve *pheromone* for the deposited marker stores introduced below and *pressure* for this sematectonic field (the cue agents read in the state of the work itself rather than in a deposited sign), following the sematectonic versus sign-based distinction of Parunak (2006). Agents are foundation-model actors that observe a region's pressure and propose a *patch*, a candidate rewrite of the region's content; coordination emerges from the shared artifact alone, with no

agent coordinating, planning, or messaging another. Two pheromone stores, applied patches kept as few-shot examples and rejected patches kept as warnings, evaporate at each tick of the system's logical clock, following pheromone evaporation Dorigo et al. (1996) and the environment-state-dynamics operator Parunak (2011), so stale patches are forgotten rather than left to mislead. The two are complementary: the models bring open-ended proposals beyond any fixed move-set, and the pressure field brings the selection criterion and capability escalation that turn proposals into progress.

This paper makes three contributions. First, it maps a foundation-model artifact-refinement loop onto the symmetric-stigmergic primitives (Section 4), including the environment-to-agent escalation arrow that discrete blackboard and co-presence systems leave implicit. Second, it proves that aligned local descent, descent in which lowering a region's pressure also lowers the total, terminates in a number of ticks bounded by initial pressure rather than by state-space size (Section 5). Third, on meeting room scheduling across a 1350-trial grid, it identifies the regime in which decentralized symmetric-stigmergic coordination outperforms hierarchical control and the smaller-instance regime in which it does not (Section 6).

2 Related Work

This work sits at the end of a single lineage running from digital pheromones to language-model agents, through five stages: the digital-pheromone abstraction of classic stigmergy; its reframing as a continuous field between agents and equations; the environment as a first-class coordination substrate; the symmetric extension that lets the environment act back on agents; and recent environment-mediated coordination for foundation-model multi-agent systems. The symmetric-stigmergic schema Parunak (2025) already unifies these stages, so we position the contribution against each in turn.

2.1 Digital Pheromones and Stigmergic Optimization

Grassé named stigmergy to explain termite construction, where deposited material attracts further deposits and complex structure emerges with no agent holding a global plan Grassé (1959). Parunak abstracted this into engineering principles for artificial multi-agent systems, identifying deposition, aggregation, and evaporation as the operators that make pheromone coordination robust and scalable Parunak (1997). Dorigo and colleagues turned the same insight into ant colony optimization, where artificial pheromone trails guide combinatorial search through positive feedback on good paths and evaporation of stale ones Dorigo et al. (1996); Dorigo and Gambardella (1997); Dorigo and Stützle (2004). The approach extends to telecommunications routing Bonabeau et al. (1998) and rests on the broader account of self-organization in biological systems Camazine et al. (2001). We inherit the deposition-and-evaporation core directly: the example bank and the rejected-patch store are pheromone fields with per-tick evaporation, and the sensed pressure field is the aggregate signal agents follow. Two task-driven differences from classic ant systems remain: the action space is open-ended natural-language patches rather than enumerated moves, and the agents reason rather than react.

2.2 Pheromone Fields Between Agents and Equations

Treating per-location pheromone variables as a continuous field places stigmergic models on a continuum between agent-based and equation-based modeling. Parunak showed analytically that pheromone coordination is intermediate between agent-based and mean-field models, with a model's position set by its pheromone parameters, notably the propagation factor Parunak (2011). Field-based middleware made the abstraction programmable: the TOTA model distributes computational fields over a network that agents sense and follow Mamei and Zambonelli (2009), and the field metaphor recurs across catalogues of biology-inspired design patterns for distributed

computing Babaoglu et al. (2006). Our pressure field is such a field, and its one-hop propagation operator instantiates that continuum control Parunak (2011): raising κ spreads activation further along the temporal chain. We keep propagation deliberately shallow, so the field stays near the agent-based end where individual patches remain the unit of action.

2.3 The Environment as a First-Class Substrate

A separate strand argued that the environment, not the agents alone, carries coordination and should be engineered as a first-class entity. The E4MAS research roadmap formalized the environment as an explicit design dimension mediating agent access to resources and to one another Weyns et al. (2015). Methodologies for engineering self-organizing systems named gradient fields as a reusable coordination pattern and catalogued the mechanisms through which global order emerges from local rules: positive and negative feedback, randomness, and repeated local interaction De Wolf and Holvoet (2005); Serugendo et al. (2005). The pattern has seen industrial use: stigmergic coordination drives manufacturing execution control Hadeli et al. (2004); Valckenaers and Van Brussel (2015), digital pheromones coordinate swarming unmanned vehicles Sauter et al. (2005), and delegate multi-agent systems route traffic anticipatorily over a pheromone substrate Claes et al. (2011). Our agents hold no model of one another and coordinate solely by reading and writing the shared artifact, so the environment is the entire coordination channel. We extend this strand with a foundation-model realization in which the environment also reallocates agent capability, made precise in the next stage.

2.4 Symmetric Stigmergy and the BDI Bridge

The symmetric extension Parunak (2025) supplies the arrow classic stigmergy lacks, letting the environment update agent state, and adds agent-side analogues of the environment's update, dynamics, and propagation operators. A single schema then spans

a continuum from ant-like agents through symmetric stigmergy to belief-desire-intention agents Bratman (1987); Rao and Georgeff (1995), building on polyagent and hierarchical-task-network stigmergy that deploys reasoning-capable agents against a stigmergic substrate Brueckner (2000); Brueckner et al. (2009). We place foundation-model actors at the symmetric-stigmergy point of that continuum and give the primitive-by-primitive mapping in Section 4. Joint-intentions accounts of teamwork require each agent to hold and reason about mutual beliefs over the others' commitments Cohen and Levesque (1991); symmetric-stigmergic coordination requires none, since the shared field carries everything the agents need to align.

2.5 Environment-Mediated Coordination for Foundation-Model Agents

Most foundation-model multi-agent systems coordinate through explicit dialogue: agents pass messages, assume roles, and decompose tasks hierarchically. AutoGen formalizes conversation among configurable agents Wu et al. (2023); MetaGPT encodes standard operating procedures into role workflows over a shared message pool Hong et al. (2024); CAMEL scripts role-play between paired agents Li et al. (2023); ChatDev structures a software team as role-based chat chains over a shared codebase Qian et al. (2024); and multi-agent debate converges instances through rounds of explicit critique Du et al. (2024). These designs scale poorly. Coordination overhead can turn multi-agent setups into a net cost on sequential tasks Kim et al. (2025), most inter-agent messages are prunable without loss Zhang et al. (2025), and performance gains follow a saturating curve in agent count Qian et al. (2025). Surveys map the same space and note that the environment-mediated, distributed-and-cooperative quadrant remains thinly populated Guo et al. (2024).

A smaller body of work coordinates foundation-model agents through the environment rather than through messages. Generative agents organize joint activity

through shared environmental co-presence and a retrievable memory stream rather than broadcast Park et al. (2023); coordination benchmarks rank environment-grounded coordination above theory-of-mind reasoning between agents Agashe et al. (2025); blackboard architectures let agents coordinate solely through a public store Han et al. (2025); agents develop implicit numerical signaling that functions like pheromone traces even without language Buscemi et al. (2025); and information-theoretic accounts measure emergence under group-level feedback with no direct communication Riedl (2025). Stigmergy proper is reaching this setting too: language-model agents have been run as pheromone-following foragers Jimenez Romero et al. (2025), digital pheromones bridge independent reinforcement learners Xu et al. (2022), and decayed pheromone maps coordinate robot swarms where conventional multi-agent reinforcement learning fails Aina et al. (2025). Norm-emergence work offers a complementary route, aligning behavior through shared normative expectations rather than fields Ren et al. (2024); Savarimuthu et al. (2024), and organization-generation work automates the role structures we instead remove Amaral et al. (2023). Within this strand, prior systems coordinate through the environment but leave its influence on agents implicit; we make that influence an explicit operator and map a complete artifact-refinement loop onto the symmetric-stigmergic primitives. Prior work suggests but does not establish that small reasoning models struggle with purely emergent decentralized coordination Valmeekam et al. (2023); Section 6 locates the regime in which environment-mediated coordination nonetheless outperforms hierarchical control.

3 Problem Formulation

Artifact refinement is decentralized descent on a sensed quality field, with agents acting only on local signals. The mapping of the elements below onto the symmetric-stigmergic primitives follows in Section 4.

3.1 Artifact, Regions, and Signals

An *artifact* is partitioned into n *regions* with content $c_i \in \mathcal{C}$ for $i \in \{1, \ldots, n\}$, where $\mathcal{C}$ is an arbitrary content space (schedule grids, strings, Abstract Syntax Tree (AST) nodes). In the scheduling domain a region is a within-day time block. Each region also carries auxiliary state es_i consisting of a per-axis pressure exponential moving average, an inhibition window expressed in logical ticks, and a provenance log of applied and rejected patches. Regions are passive subdivisions; agents are active proposers that observe a region and generate candidate patches. A patch rewrites the content of a single region and nothing else; in the scheduling domain a meeting displaced by the rewrite returns to a global pool of unscheduled meetings, from which any region's agent may later place it. That pool is the only channel through which content crosses region boundaries.

A *signal function* $\sigma : \mathcal{C} \to \mathbb{R}^d$ maps content to measurable features and is *local*: $\sigma(c_i)$ depends only on region i. Signals such as gaps, overlaps, and utilization feed the activation field that selects which regions receive proposals, and the hierarchical baseline's heuristic; they are distinct from the reported quality metric.

3.2 One Pressure Metric

All reported pressure uses a single metric, identical across every coordination strategy, so that cross-strategy comparisons measure coordination and not metric choice. On the schedule grid,

$$P(s) = 1.0 \cdot (\text{unscheduled meetings}) + 10.0 \cdot (\text{attendee overlaps}),$$

and the artifact is solved when $P(s) = 0$, that is, every meeting is placed with no double-booking. The overlap term is attributed to the block where the overlap occurs, so region pressure p_i is region i's overlap contribution; the unscheduled term is a

property of the artifact as a whole, attributed to no region, and the total pressure is the unscheduled term plus $\sum_{i=1}^{n} p_i$. Low-pressure regions are valleys where the artifact satisfies its quality constraints. Patches are accepted only if they strictly decrease the total $P(s)$, evaluated over the whole artifact, so a patch that fixes an overlap by displacing meetings into the pool is accepted exactly when the trade pays: an overlap costs ten times an unscheduled meeting.

3.3 Pheromone Stores and Evaporation

Two pheromone stores carry information across ticks, each evaporating once per logical tick and gated by a single decay-ablation flag. A *positive store* deposits each applied patch into an example bank as a few-shot example; its weight starts at 1.0, decays by a factor of 0.95 per tick, and the example is evicted below weight 0.1. A *negative store* deposits each harmful rejected patch on the artifact and injects it into later prompts for the same region as an avoid-tip, evaporating on its own schedule. The positive store propagates successful patterns across the agent population; the negative store steers proposals away from patterns the artifact has already rejected. Neither store reinforces on reuse: under uniform per-tick evaporation, weight-ranked selection reduces to recency-ranked selection.

3.4 Inhibition

A region that receives an applied patch enters an *inhibition window* of two logical ticks, during which it is not dispatched. Inhibition lets the applied change settle and forces agents toward other high-pressure regions, preventing oscillation around a single fix. All temporal dynamics, evaporation and inhibition alike, run on a single logical tick counter rather than wall-clock time, so the system is deterministic and hardware-independent.

3.5 The Locality Constraint

The constraint that distinguishes this setting from centralized optimization is that agents observe only local state. An actor dispatched to region i sees $(c_i, es_i, \sigma(c_i))$ and the pool of unscheduled meetings that fit its block, but not other regions' content c_j for $j \neq i$, the total pressure $P(s)$, or other agents' in-flight proposals. The acceptance rule of the previous subsection is evaluated by the environment runtime, never by an agent: the runtime compares $P(s)$ before and after a candidate patch, while the agents that propose patches remain local. This rules out coordinated planning: coordination must emerge from local actions aligned with global pressure reduction through the shared artifact.

4 Symmetric-Stigmergic Formalization and Method

The coordination mechanism of Section 3 instantiates the symmetric-stigmergic schema of Parunak (2025), with foundation-model agents playing the role of stigmergic actors and the method applying Parunak's three pheromone operators, aggregate, propagate, and evaporate Parunak (2011), to the sensed quality field. Classic stigmergy is asymmetric: agents change the environment, but the environment does not change agent state Grassé (1959); Parunak (1997). The symmetric extension adds the reverse arrow, so that the environment also updates agent state, and adds agent-side analogues of the environment's update, dynamics, and propagation operators. Each primitive maps to an element of the system.

4.1 Mapping the Schema

Table 1 states the correspondence. The environment $E(t)$ is the set of time-block regions of the schedule artifact; each region carries state es_i consisting of a per-axis pressure exponential moving average, an inhibition window, and a provenance log. The agent set $A(t)$ is the pool of LLM actors, each with state as_i (model identifier and sampling

band) and a location al_i (the high-pressure region it is dispatched to this tick). Agents never read each other directly: the agent adjacency relation $aa(t)$ is realized entirely through the shared artifact by a two-stage validation gate. A patch is first evaluated against a clone of the artifact, then re-evaluated against the now-current artifact, so two agents may both pass initial evaluation yet the second is rejected because the first has already changed the shared state.

A runtime ticks the logical clock, evaporates the pheromone stores, propagates activation one hop, dispatches proposals round-robin over the pressure-activated regions, and serializes writes under a fixed acceptance rule (strict pressure decrease, re-checked against current artifact state). Every rule is content-blind: the runtime never chooses what to propose, never reasons about the schedule globally, and never distinguishes one agent from another. It is the analogue of the pheromone substrate, not of a manager. A physical pheromone field is itself a consistency-enforcing medium: two ants cannot deposit on the same point, and physics serializes them. The two-stage re-evaluation is optimistic concurrency control playing that role. Placing the aggregate, evaporate, and propagate operators in a central runtime is standard digital-pheromone infrastructure Parunak (2011), with the environment carrying mediation responsibilities as a first-class component Weyns et al. (2015). The hierarchical baseline, by contrast, centralizes a reasoner that makes content decisions: coordination by a manager rather than by a rule.

4.2 The Field Operators: Aggregate, Propagate, Evaporate

The sensed field evolves through the three operators of pheromone coordination Parunak (2011).

Aggregate fuses signals into region state. Each region holds a per-axis pressure exponential moving average that combines the measurements of successive ticks, the smoothing analogue of pheromone cells averaging deposits from many agents.

Table 1 Symmetric-stigmergic primitives Parunak (2025) and their realization in this system.

Primitive	Schema role	This system
$A(t)$	Agents with state as_i, location al_i	LLM actor pool; as_i = (model, sampling band); al_i = assigned region
$E(t)$	Environment locations with state es_i	Time-block regions; es_i = pressure EMA + inhibition window + provenance
$ea(t)$	Environment adjacency	Within-day temporal chain: block b adjacent to $b\pm1$; no cross-day edges
$aa(t)$	Agent adjacency	Emergent: two-stage validation gate on the shared artifact
AL	$A(t) \to E$	Round-robin dispatch to current high-pressure regions
AM	$AL(t) \to AL(t{+}1)$	Re-dispatch each tick from the updated pressure field
ESU	$as_i \to es_j$ (deposit)	Applied patch lowers region pressure; logged in es
ESD	Environment dynamics	Pheromone evaporation of two stores (below)
ESP	Environment propagation	One-hop activation spillover along $ea(t)$ (below)
ASU	$es_i \to as_j$ (env $\to$ agent)	Band and model escalation on pressure stagnation
ASD	Agent dynamics	Per-band randomized sampling (temperature, top-p)
ASP	$as_i \to N(as_j)$	Example bank (environment-mediated): deposit on applied patch, evaporate, weight-ranked reuse

Propagate (ESP) spreads activation along the temporal chain, addressing the intrinsic structure of the scheduling domain: adjacent time blocks are where a conflicting meeting can be shifted. The environment adjacency $ea(t)$ links each within-day time block to its immediate temporal neighbors, with no edges across days. Each tick the environment runtime computes an effective activation pressure

$$g_i = p_i + \kappa \sum_{j \in N(i)} p_j, \qquad \kappa = 0.5,$$

from regions' own pressures p_j, applied exactly one hop per tick, so a high-pressure block recruits proposals in its idle temporal neighbors without those neighbors being individually above threshold. Because patches are strictly within-block, a meeting moves between blocks in two steps through the unscheduled pool: the hot block's

own agent evicts a conflicting meeting (accepted because an overlap costs ten times an unscheduled meeting), and a recruited neighbor, seeing the meeting among the unscheduled candidates for its block, places it. The spillover affects only activation, which regions receive LLM proposals; the reported pressure metric is computed from regions' own state and stays the sum of own pressures, unaffected by spillover. Inhibited regions contribute spillover but are not themselves dispatched. This is single-application propagation, one hop per tick from own pressures, not multi-hop accumulated diffusion. The operator is flag-gated, and Section 6 reports a no-propagation ablation arm against the full system.

Evaporate (ESD) discards stale information from the two pheromone stores of Section 3, the positive example bank and the negative rejected-patch store, following pheromone evaporation Dorigo et al. (1996) and the ESD operator Parunak (2011). Both evaporate once per logical tick, gated by a single decay-ablation flag. Evaporation is truth maintenance in constant time Parunak (2011): stale patches are forgotten without any logical bookkeeping. Because the schedule is the modified task structure that agents sense, the pressure field is a sematectonic field rather than a deposited marker Parunak (2006), and it is exactly the environment state es_i of the schema, not a separate quantity.

4.3 The Environment-to-Agent Operators: Escalation and Example Propagation

The agent-state-update operator ASU is realized as escalation driven by pressure stagnation, where stagnation means no pressure improvement above 0.01. The sampling band escalates every 7 stalled ticks (Exploitation $\rightarrow$ Balanced $\rightarrow$ Exploration); the model escalates every 21 stalled ticks (qwen2.5 0.5b $\rightarrow$ 1.5b $\rightarrow$ 3b), resetting the band. This is the environment acting on agent capability: the sensed field, through its own lack of progress, reallocates reasoning capacity.

The agent-state-propagation operator ASP is realized by the example bank: an applied patch propagates as a few-shot example available to subsequent proposals across the population. The realization is mediated rather than direct: the bank is carried by the environment, and what spreads is deposited content, not agent state itself, since no agent's model tier or sampling band propagates to another. Under uniform per-tick evaporation, weight-ranked selection is recency-ranked; the bank carries no use-count reinforcement and follows no regional-similarity trail, so we claim deposit-and-evaporate propagation only, not positive-feedback trail-following.

4.4 Alignment and the Potential-Game Observation

Decentralized descent succeeds only when local incentives track the global objective, because the locality constraint of Section 3 forbids agents from observing global state. We call a pressure system *aligned* when reducing a region's own pressure cannot raise the total.

Definition 4.1 (Pressure Alignment)**.** A pressure system is *aligned* if, for any region i, state s, and action a_i with $s' = s[c_i \mapsto a_i(c_i)]$,

$$P_i(s') < P_i(s) \implies P(s') < P(s).$$

Alignment holds automatically when the pressure is *separable*, each P_i depending only on c_i so that $P(s) = \sum_i P_i(s)$, and more generally under bounded coupling: if modifying region i changes every other region's pressure by at most ϵ, then a local reduction exceeding $(n-1)\epsilon$ still lowers the total. In the implementation alignment also holds by construction, because the acceptance rule itself evaluates the artifact's total pressure: the runtime applies a patch only if P strictly decreases, so no accepted patch can trade a local gain for a global loss. The definition matters for variants whose gates check only local pressure; the separability analysis for this domain is in Appendix B.

Under alignment the artifact pressure $P(s)$ is a potential function for the game whose players are regions, whose strategies are content choices, and whose common potential is $\Phi(s) = P(s)$. Any sequence of pressure-reducing moves is then a sequence of improving moves on Φ, so the finite-improvement property of potential games Monderer and Shapley (1996) applies: greedy local descent terminates at a state where no region can reduce its pressure below the activation threshold. Section 5 bounds the convergence time.

4.5 The Tick Loop

Each tick measures the field, propagates activation, dispatches actors against the high-pressure regions, validates proposals in two stages against the shared artifact, applies the survivors, and updates the pheromone stores. As throughout, the loop runs on the logical tick clock (Section 3).

Algorithm 1: **Pheromone-Field Tick**

Input: state s^t; signal functions $\{\sigma_j\}$; pressure functions $\{\phi_j\}$; actor pool $\{a_k\}$; parameters $(\tau_{\text{act}}, \kappa, \tau_{\text{inh}})$

1. Measure and aggregate.
For each region i: $\boldsymbol{\sigma}_i \leftarrow \sigma(c_i)$; update the pressure EMA in es_i; let $p_i \leftarrow P_i(s^t)$.

2. Propagate (ESP).
For each region i: $g_i \leftarrow p_i + \kappa \sum_{j \in N(i)} p_j$ (one hop, from own pressures).

3. Dispatch.
Activate regions with $g_i \geq \tau_{\text{act}}$ and not inhibited; round-robin assign actors to them.
Each actor a_k observes only local $(c_i, es_i, \boldsymbol{\sigma}_i)$ and proposes a patch δ.

4. Two-stage validation.
Stage 1: evaluate each δ on a clone of the artifact; keep those that reduce the total pressure P.
Stage 2: re-evaluate survivors against the now-current artifact, so a patch is rejected if an earlier-applied patch this tick has already changed the shared state.

5. Apply and update stores.
For each accepted patch on region i: $c_i \leftarrow \delta(c_i)$; deposit δ in the positive example bank; inhibit i for τ_{inh} ticks.
For each harmful rejected patch: deposit it in the negative store.
Evaporate both stores once.

Return updated state s^{t+1}.

The loop has three properties. Coordination is *stigmergic*: actors read and write only the shared artifact and never message one another, so the agent adjacency $aa(t)$ emerges entirely from stage-two validation. Concurrency is *bounded*: distinct high-pressure regions are addressed in the same tick, with stage-two validation resolving same-tick conflicts and inhibition preventing repeated modification of one region; the same gate also contains model error, so a confabulated or ill-formed patch fails validation and leaves the artifact untouched, and a hallucinating agent costs a proposal, never a corrupted artifact. Capability is *adaptive*: when pressure stalls, the environment escalates sampling band and then model tier through the ASU operator described above.

Termination is economic rather than logical: the loop halts when every region's effective pressure falls below the activation threshold, at which point no actor is dispatched, or when a tick budget is reached. Activity ceases when the field flattens, not when an external goal is declared met; we impose a maximum tick count to bound computation.

4.6 Position on the Modeling Continuum

The system classifies on the continuum of Figure 3 of Parunak (2025), which spans equation-based mean-field models Parunak (2011), classic stigmergy, and belief-desire-intention agents Bratman (1987); Rao and Georgeff (1995), at the symmetric-stigmergy point between stigmergic agents and BDI agents. Each LLM actor brings BDI-grade reasoning to a single patch proposal, yet the actors are deployed stigmergically: they hold no beliefs, desires, or intentions about one another and coordinate only through the sensed field. Of the two arrows that asymmetric stigmergy lacks, ASU is realized directly, the environment updating agent capability when the field stalls, and ASP only in environment-mediated form, so the placement at the intermediate point rests chiefly on ASU.

5 Theoretical Analysis

Aligned local descent terminates, with a bound that scales with initial pressure rather than with the size of the state or action space.

5.1 Convergence Under Alignment

Lemma 5.1 (Termination of Aligned Descent) *Let the pressure system be aligned with ϵ-bounded coupling over n regions, and let $\delta_{\min}$ be the minimum local pressure reduction from any applied patch, with $\delta_{\min} > (n-1)\epsilon$. Call a tick* productive *if at least one patch is applied in it. Then from any initial state s_0 with pressure $P_0 = P(s_0)$, greedy local descent reaches a state where every region is below the activation threshold within*

$$T \leq \frac{P_0}{\delta_{\min} - (n-1)\epsilon}$$

productive ticks.

Proof. The total pressure $P(s)$ is non-negative and, by alignment, each applied patch lowers it by at least $\delta_{\min} - (n-1)\epsilon > 0$ (Appendix C). A non-negative quantity that decreases by a fixed positive amount each productive tick reaches its floor within the stated bound, at which point no region exceeds the activation threshold and no actor is dispatched. □

The bound counts productive ticks only; nothing bounds the unproductive ticks in which every proposal fails validation, and such stalls occur in practice (Section 7.2). The escalation operator of Section 4 exists to end them, and the experiments measure them; wall-clock convergence is therefore an empirical question, not a guaranteed one. What the lemma establishes is the property that matters for decentralized coordination: descent length tracks how far the artifact starts from acceptable, not how large its content space is. In the scheduling domain pressures are integer multiples of 1.0, every accepted patch strictly decreases P, and the per-region metric is separable, so $\epsilon = 0$ and $\delta_{\min} = 1.0$ (Appendix B); the bound applies.

5.2 Remarks

Quality of the terminal state. Because each region is below the activation threshold τ_{act} at termination, the terminal *regional* pressure is at most $n \cdot \tau_{\text{act}}$; a globally attributed term, such as the scheduling domain's unscheduled count, is not bounded by any region's threshold. The terminal state is a local minimum of P, not necessarily its global minimum, and the legality stalls of Section 7.2 are terminal states retaining exactly the global term. On the scheduling domain the terminal state is exact ($P = 0$) whenever the problem is solved.

Independence from agent count. Per-tick cost is dominated by measuring and scoring the n regions and ranking the proposed patches; adding actors raises throughput, the number of patches proposed per tick, without adding coordination cost, because actors exchange no messages and share state only through the artifact. This is the $O(1)$ coordination overhead of stigmergy: per-agent coordination work does not grow with the population, where manager delegation grows linearly in workers and all-to-all messaging quadratically.

Parallel speedup. When several accepted patches in a tick target disjoint regions, their pressure reductions are additive, so the effective descent rate rises with the number of non-conflicting patches applied per tick. Stage-two validation caps this: same-region or same-tick conflicting patches are rejected, so the speedup is bounded by how many distinct high-pressure regions the field exposes.

6 Experiments

The pheromone-field coordination of Section 4.5 is evaluated on meeting room scheduling, assigning meetings to rooms over a five-day horizon to remove unscheduled meetings and attendee overlaps. The domain supplies a sensed pressure field with measurable success and difficulty that scales with problem size, while staying small enough that hierarchical and conversational baselines remain feasible to run. The question is where

decentralized symmetric-stigmergic coordination outperforms hierarchical control, and where it does not.[1]

6.1 Setup

6.1.1 Task: Meeting Room Scheduling

We generate scheduling problems with varying difficulty:

Difficulty	Rooms	Meetings	Pre-scheduled
Easy	3	20	70%
Medium	5	40	50%
Hard	5	60	30%

Table 2 Problem configurations. Pre-scheduled percentage indicates meetings already placed; remaining meetings must be scheduled by agents.

Each schedule spans 5 days with 30-minute time slots (8am–4pm). Regions are 2-hour time blocks (4 blocks per day $\times$ 5 days = 20 regions per schedule). A problem is "solved" when all meetings are scheduled with zero attendee overlaps within 50 ticks.

One authoritative pressure metric. Every number in the tables below is the single grid metric of Section 3, $P = 1.0 \cdot (\text{unscheduled}) + 10.0 \cdot (\text{overlaps})$, applied identically to all five strategies; the sensor signals that feed activation (gap, overlap, utilization) never enter it.

Alignment. The domain satisfies the alignment condition of Section 4 on both counts: the per-region metric is separable ($\epsilon = 0$, since a patch rewrites one block and the unscheduled pool is charged to no region), and the acceptance gate evaluates the full metric. Appendix B reports the empirical check.

[1] An earlier draft reported parity between pheromone-field and hierarchical control on a smaller pilot; the full grid showed a regime-dependent gap. A code audit found two defects in the measurement path: a system-prompt confound that advantaged some baselines under a Latin-square assignment, and a pressure-metric mismatch across strategies. Both are fixed, the strategies now share one authoritative pressure metric, and every result below comes from the regenerated 1350-trial grid.

6.1.2 Baselines

We compare five coordination strategies, all using identical Large Language Models (LLMs) (`qwen2.5:0.5b/1.5b/3b` via Ollama) to isolate coordination effects:

Pheromone-field (ours): Full system with pheromone evaporation (example bank 0.95/tick, rejected-patch store), inhibition (2 logical ticks), one-hop ESP propagation ($\kappa = 0.5$), and two-stage validation. Includes band escalation (Exploitation $\rightarrow$ Balanced $\rightarrow$ Exploration, every 7 stalled ticks) and model escalation (0.5b $\rightarrow$ 1.5b $\rightarrow$ 3b, every 21 stalled ticks).

Conversation: AutoGen-style multi-agent dialogue where agents exchange messages to coordinate scheduling decisions. Agents discuss conflicts and propose solutions through explicit communication.

Hierarchical: Single agent selects the highest-pressure time block each tick, proposes a schedule change, and validates before applying (only accepts pressure-reducing patches). Uses the identical base prompt to pheromone-field; pheromone-field prompts additionally carry the few-shot examples and avoid-tips injected from the two pheromone stores. The differences are: (1) greedy region selection always targets the hardest region, and (2) sequential execution processes one region per tick. This represents centralized, quality-gated control.

Escalation across strategies. Every strategy, baselines included, receives model escalation on the same trigger, 21 stalled ticks stepping 0.5b $\rightarrow$ 1.5b $\rightarrow$ 3b, so no strategy is capability-starved relative to another. Band escalation is part of the pheromone-field mechanism only; the conversation baseline uses fixed sampling (T=0.3, top-p=0.9) suited to structured dialogue. The comparison therefore does not withhold capability from the baselines: what differs is whether escalated capability arrives at field-selected regions or at the single region a greedy controller keeps choosing.

Sequential: Single agent iterates through time blocks in fixed order, proposing schedule changes one region at a time. No parallelism, pressure guidance, or patch validation, so it applies any syntactically valid patch regardless of quality impact.

Random: Selects time blocks uniformly at random and proposes schedule changes; like sequential, it validates nothing and applies any syntactically valid patch.

Note on concurrency: Pheromone-field addresses several distinct high-pressure regions in the same tick, each proposal validated against the shared artifact by the two-stage gate; hierarchical addresses one region per tick. This asymmetry follows from the coordination paradigm, not from an implementation choice: hierarchical control selects a region, delegates, and validates before proceeding, and dispatching to several regions at once would require the work-distribution, conflict-resolution, and aggregation protocols that define a different architecture. When hierarchical's single patch is rejected, the tick makes no progress; when one of pheromone-field's concurrent patches is rejected, the others can still apply.

Model choice rationale: We deliberately use small models (0.5b–3b parameters) so that coordination, not model capability, drives the results. Whether frontier models preserve the relative ranking is an empirical question this study does not settle (Section 7).

6.1.3 Metrics

- **Solve rate**: Percentage of schedules reaching all meetings placed with zero overlaps within 50 ticks.
- **Ticks to solve**: Convergence speed for solved cases.
- **Final pressure**: Remaining unscheduled meetings and attendee overlaps for unsolved cases, on the authoritative metric
- **Token efficiency**: Total prompt and completion tokens consumed per trial and per successful solve

6.1.4 Implementation

Software: Rust implementation with Ollama for local inference. **Grid**: 5 strategies × {1, 2, 4} agents × {easy, medium, hard} × 30 trials = 1350 trials, all with a 50-tick budget. **Determinism**: all temporal dynamics run on the runtime's logical tick clock, so results are deterministic and hardware-independent; each trial's problem is generated from a seed of trial × 1000 + agent_count, so a given trial faces an identical problem across all strategies. The ablation study is a separate 9-arm matrix (Section 6.3). Escalation parameters are specified in Section 6.5; the full protocol is in Appendix A.

6.2 Main Results

Across the 1350-trial grid, aggregate solve rate stratifies the five strategies:

Strategy	Solved/N	Rate	95% Wilson CI
Pheromone-field	118/270	43.7%	[37.9, 49.7]
Conversation	25/270	9.3%	[6.4, 13.3]
Hierarchical	25/270	9.3%	[6.4, 13.3]
Sequential	4/270	1.5%	[0.6, 3.7]
Random	2/270	0.7%	[0.2, 2.7]

Table 3 Aggregate solve rates across the grid (1350 trials, 270 per strategy). Omnibus test across all five strategies: $\chi^2 = 301.5$, $df = 4$, $p < 0.001$.

The pheromone-field advantage over hierarchical control holds in the regime where strict-greedy hierarchical region selection collapses into the *rejection loop* (defined in Section 7.2); on the smallest instances, where that loop does not trigger, the gap narrows (Section 6.6).

Pheromone-field leads aggregate solve rate, solving 4.7× as many instances as the next-best baseline (conversation), with effect sizes reported per comparison in Section 6.6. **Conversation is intermediate**, solving 28% of easy problems and none of the medium and hard. **Hierarchical, sequential, and random collapse** on this

grid: hierarchical reaches 9.3% aggregate, near random (0.7%), because of the rejection loop. In this regime, the overhead of centralized, strictly-gated control is a net cost, counter to the assumption that explicit hierarchical coordination should beat implicit coordination.

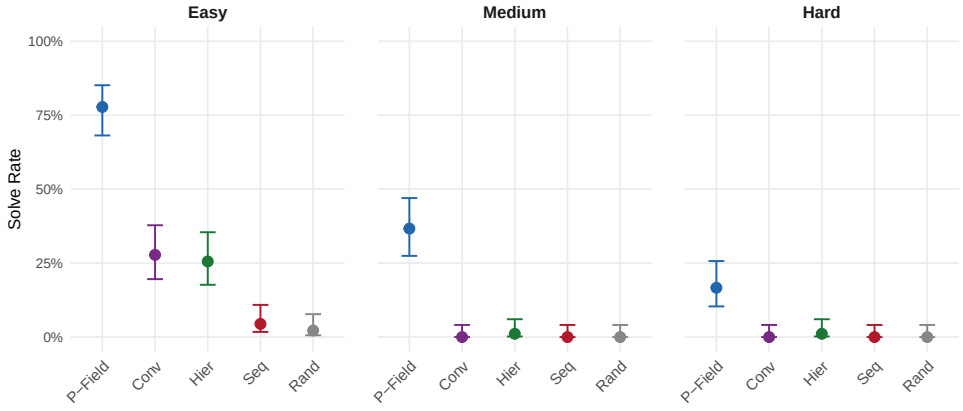

Fig. 1 Strategy comparison by difficulty level. Error bars show 95% Wilson Confidence Intervals. On medium and hard problems only pheromone-field retains an appreciable solve rate; every other strategy falls to at most one solved instance per 90.

6.3 Ablations

The ablation runs on medium problems, not easy: at the easy solve rate the base mechanism already succeeds on most instances, leaving no room to measure what each component adds. Medium leaves the full system at a 40% solve rate, with headroom in both directions.

The 9-arm matrix crosses evaporation, inhibition, ESP propagation, and few-shot examples: four single-feature removals, three pairwise removals, the full system, and an all-off baseline.

No single removal moves the solve rate significantly: every arm's Fisher exact p against the full system exceeds 0.19, and the all-off baseline (36.7%) is statistically indistinguishable from the full system (40.0%, $p = 1.0$). The large advantage of the full

Configuration	Evap	Inhib	ESP	Examples	Solve Rate
Full	✓	✓	✓	✓	40.0%
No evaporation	×	✓	✓	✓	33.3%
No inhibition	✓	×	✓	✓	30.0%
No examples	✓	✓	✓	×	60.0%
No propagation	✓	✓	×	✓	36.7%
No evap., no inhib.	×	×	✓	✓	50.0%
No evap., no examples	×	✓	✓	×	43.3%
No inhib., no examples	✓	×	✓	×	33.3%
Baseline (all off)	×	×	×	×	36.7%

Table 4 Ablation results (30 trials per arm, medium difficulty). The `no_propagation` arm isolates the one-hop ESP operator of Section 4. No arm differs from the full system at $p < 0.05$ (Fisher exact).

system over the conversational and hierarchical baselines (Section 6.2) therefore does not come from the four pheromone enhancements. It is structural: decentralized field-based concurrent dispatch with strict two-stage validation, the part that outperforms hierarchical control, addresses many high-pressure regions per tick while admitting only strictly improving patches. Those two mechanisms (concurrent field-based dispatch and strict validation) are present in every arm, the all-off baseline included, so removing the enhancements layered on top of them changes little.

Pooling across the design sharpens the same reading. Each feature's marginal effect, computed as the solve rate with it enabled minus the rate with it disabled across all nine arms (roughly 135 trials per side), is small and non-significant: evaporation -0.8 points ($p = 0.90$), inhibition $+5.2$ ($p = 0.45$), propagation $+4.8$ ($p = 0.55$), examples -5.3 ($p = 0.39$) (Figure 2). At 30 trials per arm the design has power to detect only large effects; contributions of this size would need several hundred trials per arm to separate from zero: no temporal-dynamics component is individually decisive at this scale, and we do not claim a precisely measured contribution for any of them.

The one consistent signal is that the example bank does not pay for itself. Removing it raises the solve rate to 60%, the best of any arm, and roughly halves token cost, because every prompt otherwise carries the few-shot examples; the deposited examples appear

to mislead the small models more often than they help, a positive-pheromone analogue of a stale trail. The ESP propagation arm is null, as predicted: a pooled marginal effect of +4.8 points ($p = 0.55$), with the no-propagation arm (36.7%) indistinguishable from the full system (40.0%). Section 7.1 reads this null, and the win that accompanies it, as a lower bound: local decomposability at this instance size leaves no cross-region structure to exploit. Component removals leave time-to-solution unchanged as well: among solved trials, mean ticks differ by less than one across the evaporation, inhibition, and propagation contrasts.

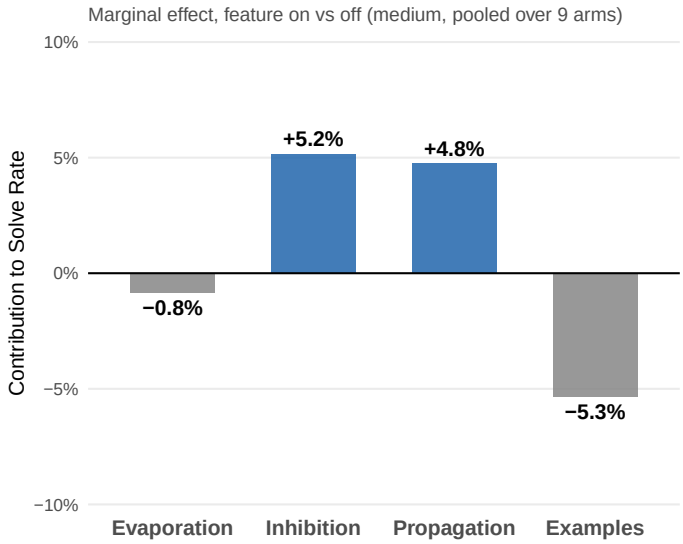

Fig. 2 Marginal effect of each component, solve rate with the feature enabled minus the rate with it disabled, pooled across all nine arms (medium difficulty). All four effects are small and non-significant; the example bank's is negative.

6.4 Scaling Experiments

Pheromone-field solve rate across 1, 2, and 4 agents on easy difficulty tests the agent-count independence of Section 5:

Agents	Solved/N	Rate	95% CI
1	25/30	83.3%	[65.5, 92.9]
2	22/30	73.3%	[55.6, 85.8]
4	23/30	76.7%	[59.1, 88.2]

Table 5 Pheromone-field scaling from 1 to 4 agents (easy difficulty, 30 trials each).

Solve rate holds from one to four agents, with all three confidence intervals overlapping (83.3%, 73.3%, 76.7% at one, two, and four agents): coordination overhead is $O(1)$ because actors share state only through the artifact.

6.5 Band and Model Escalation

Escalation is the *ASU* operator of Section 4 in operation: when the sensed field stops improving, the environment reallocates agent capability. It runs at two levels, both keyed to pressure stagnation (no improvement above 0.01) on the logical tick clock.

Band escalation adjusts sampling within a model. After 7 stalled ticks the sampling band steps from Exploitation through Balanced to Exploration, widening the proposal distribution as progress stalls (parameter ranges in Appendix A, Table A1).

Model escalation adjusts capability across the model chain. After 21 stalled ticks the model steps 0.5b $\rightarrow$ 1.5b $\rightarrow$ 3b, and the band resets to Exploitation so the larger model first exploits before widening its search. The escalation criterion is the pressure signal alone: a stalled field, not an explicit difficulty estimate, triggers the recruitment of more capable reasoning. The grid runs with escalation enabled throughout, and the *ASU* arrow's contribution is visible in which model produces each solve: the share of solved trials that required stepping past the smallest model rises with difficulty, from 27% on easy to 33% on medium to 67% on hard, where ten of fifteen solves occurred only after escalation from the 0.5b to the 1.5b model. No solved trial required the 3b model, so escalation contributes capability up to the middle tier, beyond which the remaining failures are capability-saturated or held by the legality stall of Section 7.2

rather than by a model too small. A direct escalation-off arm on the hard problems (pheromone-field, two agents, 30 grid-seed trials, escalation disabled at both levels) solved 1/30 against 3/30 with escalation enabled, in the predicted direction but below detectability at this sample size (exact McNemar $p = 0.63$). We therefore read the escalation evidence as observational, from the model-by-solve composition above, rather than as a controlled causal estimate.

6.6 Difficulty Scaling and the Winning Regime

Solve rate by difficulty locates the regime in which each ordering holds:

Difficulty	Pheromone-field	Conversation	Hierarchical	Sequential/Random
Easy	77.8% (70/90)	27.8% (25/90)	25.6% (23/90)	4.4% / 2.2%
Medium	36.7% (33/90)	0% (0/90)	1.1% (1/90)	0% / 0%
Hard	16.7% (15/90)	0% (0/90)	1.1% (1/90)	0% / 0%
Total	43.7% (118/270)	9.3% (25/270)	9.3% (25/270)	1.5% / 0.7%

Table 6 Solve rate by difficulty (90 trials per strategy per difficulty, 270 per strategy total). See Table 3 for confidence intervals.

The breakdown defines the winning regime:

1. **The regime where pheromone-field wins is the hard regime.** As difficulty rises, the baselines fall to zero solve rate while pheromone-field retains a non-zero rate (36.7% medium, 16.7% hard); this is the regime in which strict-greedy hierarchical region selection collapses into the rejection loop of Section 7.2.
2. **On the easiest instances the gap is narrower.** Where the rejection loop does not trigger, conversation and hierarchical solve a share of problems (27.8% and 25.6%), and the pheromone-field margin over them is a finite multiple (2.8× over conversation) rather than the categorical gap seen on harder instances. Pheromone-field's own easy misses share a single signature, descent to exactly one unscheduled

meeting ($P = 1.0$) that no strictly improving single-region patch can place (all 20 of its easy misses end there); Section 7.2 traces these to the legality stall.

3. **Effect sizes.** On easy problems, pheromone-field's advantage over every baseline is large by Cohen's h: 1.05 (conversation), 1.10 (hierarchical), 1.74 (sequential), and 1.86 (random), all above the conventional large-effect threshold of 0.8.

6.7 Convergence Speed and Solution Quality

Among solved problems pheromone-field converges within budget, and on every difficulty it leaves a lower-pressure schedule than either baseline. Table 7 reports both: ticks to solution for solved cases, and final pressure averaged over solved and unsolved trials.

	Ticks to solution			**Final pressure**		
Strategy	**Easy**	**Medium**	**Hard**	**Easy**	**Medium**	**Hard**
Pheromone-field	17.2 ($n = 70$)	23.4 ($n = 33$)	36.5 ($n = 15$)	0.2	1.9	4.7
Conversation	30.4 ($n = 25$)	—	—	1.8	18.5	40.6
Hierarchical	26.1 ($n = 23$)	—	—	2.2	13.0	27.8

Table 7 Convergence speed and final pressure by difficulty. Ticks to solution is over solved cases only; dashes indicate fewer than two solved cases for that cell (hierarchical solves one medium and one hard instance; conversation none). Final pressure is averaged over solved and unsolved trials (lower is better); sequential and random, which apply patches without validation, average final pressures in the hundreds to low thousands and are omitted.

On easy problems pheromone-field converges faster than both baselines (1.8× faster than conversation, 1.5× faster than hierarchical), the hierarchical ratio resting on its small solved count ($n = 23$). It takes somewhat longer on the hard problems it solves than on the medium ones (36.5 vs 23.4 ticks), both within the 50-tick budget, so among solved problems the limiter is whether a problem solves at all, not how long it takes. The unsolved-problem limiters lie elsewhere: model capability, where escalation history shows progress arriving with larger tiers (Section 6.5), and the legality stall of Section 7.2, where descent halts at a local minimum no larger model could leave.

Pheromone-field also reaches lower final pressure than both baselines at every difficulty (on hard problems, 4.7 versus 27.8 for hierarchical and 40.6 for conversation). The average includes unsolved trials, where a baseline that never applies a patch retains near-maximal pressure, so it partly measures how often a strategy fails to start. The directional conclusion is robust to that caveat: even on problems it does not fully solve, pheromone-field leaves a lower-pressure partial schedule.

6.8 Token Efficiency

Whether pheromone-field's solve rate costs extra tokens depends on whether cost is counted per trial or per solve. Per trial it uses more, because it dispatches several concurrent proposals per tick:

Strategy	**Prompt**	**Completion**	**Total**
Pheromone-field	916k	98k	1014k
Conversation	184k	7k	192k
Hierarchical	34k	5k	40k
Sequential	47k	5k	52k
Random	48k	5k	54k

Table 8 Average token usage per trial.

Per successful solve, however, the gap largely closes. Normalizing total tokens by solves gives pheromone-field 2.32M per solve (273.8M over 118 solves) against conversation's 2.07M (51.8M over 25), a 1.1× difference rather than the 5.3× of per-trial cost. Token efficiency is therefore not where pheromone-field wins; its advantage is coverage. Conversation's solves are confined to easy problems, so its low per-solve cost buys solutions only where every strategy does reasonably well, whereas pheromone-field's tokens buy solutions at every difficulty, including the medium and hard instances no baseline solves at all.

Cost control through escalation. Escalation (Section 6.5) acts as implicit cost control: trials begin on the smallest model with exploitation sampling, recruiting larger models and wider sampling only when pressure stalls. Easy problems that solve quickly rarely escalate, so per-trial cost concentrates on harder instances: 345k tokens per easy trial, 1.0M per medium, and 1.7M per hard, the rise tracking the escalation that harder instances trigger.

7 Discussion

Decentralized, field-based concurrent dispatch with strict two-stage validation outperforms hierarchical control in locally-decomposable domains, where centralized greedy selection collapses into the rejection loop. The way that collapse happens is itself evidence for the symmetric extension.

7.1 Local Decomposability Explains the Win, the Propagation Null, and the Lower Bound

At the scale tested, the calendar is locally decomposable: fixing a conflict in one time block rarely creates one in a distant block. This single property, together with the general cost of centralized coordination, explains all three results.

It is why decentralized greedy descent wins, through three compounding effects. A global plan buys nothing when conflicts do not propagate across regions, consistent with the finding that language-model agents read environmental state better than they reason about other agents Agashe et al. (2025). Centralized coordination meanwhile spends budget on coordination rather than the problem: a manager-worker protocol issues several model calls per applied patch, attempting fewer patches per tick, the coordination-overhead cost that large-scale studies report (Section 2). And concurrent dispatch to several high-pressure regions lets progress happen wherever the artifact

yields, so no single hard region stalls the whole run. The advantage is not tuning but structure.

It is why one-hop ESP propagation is null. Activation spillover helps only when relieving one region shifts work to a neighbor, which at this instance size and with a separable metric is rare. The ablation confirms it (Section 6.3): the propagation arm is statistically indistinguishable from the full system. This is the predicted outcome, not a failure of the operator, which we implement to complete the schema mapping (Section 4).

It is why the result is a lower bound, not a ceiling. Propagation must bite where this study does not test: multi-hop diffusion, region relations beyond a single temporal chain, and combinatorial complexity high enough that relieving one region cascades conflict into others. That swarm-scale regime is future work; the present finding maps where the structure-exploiting version would live rather than showing the domain to be impoverished, and Section 7.2 traces the win to the two failure modes that separate the paradigms.

7.2 Two Failure Modes: Rejection Loops and Legality Stalls

Each coordination paradigm fails through its own characteristic mechanism, and the two mechanisms locate what escalation can and cannot repair.

Hierarchical control collapses on the harder instances through the *rejection loop*. Strict-greedy control always selects the highest-pressure region, but a region is highest-pressure precisely because it resists improvement. When the proposed patch fails validation, the region stays highest-pressure and is selected again, so the controller re-attacks the one region it cannot fix while the rest of the artifact goes untouched. The rate of this pathology is reported with the failure statistics: 13% of hierarchical runs apply no patch at all, and roughly 94% of the manager's proposals are rejected by the validation gate.

The loop is a selection pathology, not a capability deficit, and the protocol makes that distinction measurable. The hierarchical baseline shares the system's model-escalation trigger (Section 6), so when it stalls it eventually proposes with the largest model in the chain, and it still re-attacks the same region: escalated capability arrives at the same greedy choice. Two things are needed to escape, and the symmetric system supplies both. Field-based concurrent dispatch keeps progress flowing on the regions that are tractable while the hard region waits out its inhibition window, and the environment-to-agent escalation operator of Section 4 reallocates capability where the field has stalled, the share of solves requiring escalation rising with difficulty (Section 6.5). The hierarchical failure is therefore not a tuning accident but a structural consequence of coupling all capability, however escalated, to a single greedy selection rule; the experiments make that collapse the empirical case for coordinating selection through the field.

Pheromone-field coordination fails through a different mechanism, the *legality stall*, and its unsolved easy instances share one signature: the run descends to a single unscheduled meeting with zero overlaps ($P = 1.0$) and stays there (all 20 of its unsolved easy instances end at exactly $P = 1.0$). Placing the last meeting requires first moving a placed meeting that blocks its attendees, but every legal first step of that rearrangement is non-improving: evicting the blocker to the pool raises P by one, and an in-block swap that places one meeting while evicting another leaves P unchanged, so the strict-decrease gate rejects both. This is greedy improving-move dynamics terminating at a local minimum of the potential rather than its global minimum Monderer and Shapley (1996), realized in the gate. Escalation cannot repair it, because no amount of proposal quality makes a strictly improving single-region patch exist: in the stalled trials the proposal stream continues at full rate with the model escalated to the top of the chain, and the rejected proposals are dominated by exactly the swap and eviction moves that would begin the two-step escape (33,821 rejected proposals at $\Delta = 0$ and 50,177 at $\Delta = +1$ across the grid, together 47% of all rejections). The gate that contains

model error (Section 4.5) is the same rule that forbids these neutral intermediate states; containment and greed are one mechanism, and the legality stall is its price.

The taxonomy is then simple, and it turns on what escalation can repair. Rejection loops are selection failures, cured by coordinating selection through the field rather than by escalation; legality stalls are action-space failures, which nothing in the present operator set cures: an atomic evict-and-place move spanning two regions would clear the plateau under the same strict gate, at the cost of widening the single-region patch to a two-region transaction (Section 7, future work). The capability stall between them is the one failure escalation does resolve, a proposal failure the *ASU* arrow answers by recruiting a larger model.

7.3 Limitations

Propagation and scale bound the regime, not the contribution. The propagation operator is one hop, and the instances are sized to keep hierarchical and conversational baselines feasible, the opposite of the high-combinatorial regime where stigmergy is usually argued to excel. Section 7.1 reads both as a lower bound on a locally-decomposable regime, mapping where multi-hop diffusion at swarm scale would have to be tested.

Absolute solve rates are bounded by capability and by the gate. Even the pheromone-field strategy leaves many hard instances unsolved (16.7% solved on hard); with 0.5b–3b models, convergence analysis points to model capability as one limit on hard problems, and the legality stall of Section 7.2 as a second limit that larger models would not remove. Larger models would plausibly raise absolute rates on capability-bound instances, an effect this study does not measure.

Evidence is from a single domain. Results on meeting room scheduling may not transfer to domains lacking measurable local quality signals or locality. The mechanism

itself, the sensed field, evaporation, propagation, and two-stage validation, is domain-agnostic, but only the pressure function carries domain knowledge, and a mis-specified pressure function invites Goodhart-style gaming, where agents reduce the measured pressure without improving true quality.

7.4 Societal Considerations

Decentralized coordination shifts three deployment concerns relative to hierarchical control, each addressable. Accountability diffuses across agents, so regulated deployments need an audit log of patch provenance, pressure at proposal time, and validation result, which the region-state provenance log already records. Metric gaming is a standing risk because agents optimize the pressure function rather than true quality, so the function should span multiple orthogonal axes and be audited against human judgment. Explainability is mechanistic, not causal: the system records that a patch reduced a high-pressure region, not why that patch was chosen, suiting cheaply verified outcomes and weaker where human justification is required. These are deployment requirements around the mechanism, not changes to it.

7.5 Future Work

- **Richer environment propagation.** Extend the one-hop operator to the multi-hop diffusion and non-temporal region relations of Section 7.1, where propagation should bite, and locate the best operating point on the agent-to-equation continuum.
- **Escaping legality stalls.** Either widen the action space to atomic two-region moves, which pass the strict gate unchanged, or relax acceptance to admit non-worsening patches under a cycle guard; the first preserves the gate's containment property, the second trades it for mobility.
- **Scale and capability.** Test at swarm-scale problem sizes and with larger models, to separate the regime boundary from the small-model ceiling.

- **Learned and multi-artifact pressure.** Learn pressure functions from solution traces, and extend to coupled artifacts where a patch in one shifts pressure in another.

8 Conclusion

We instantiated the symmetric-stigmergic schema of Parunak (2025) in a foundation-model multi-agent system, mapping a full artifact-refinement loop onto its primitives and realizing the environment-to-agent arrow that classic asymmetric stigmergy lacks. The system places reasoning agents at the symmetric-stigmergy point of the modeling continuum, between ant-like agents and belief-desire-intention agents: each actor brings open-ended reasoning to a single patch, yet the actors coordinate only through the shared field, holding no beliefs or intentions about one another. Foundation models supply the open-ended proposals that classic stigmergy could never enumerate; the symmetric schema supplies the objective criterion that ranks them and the escalation that recruits capability when the field stalls.

The empirical contribution is a scoped one. On meeting room scheduling across a 1350-trial grid, decentralized symmetric-stigmergic coordination outperforms hierarchical control in the regime where strict-greedy control collapses into the rejection loop, and the advantage narrows on the smallest instances where that loop does not trigger. That the baseline shares the system's model-escalation trigger and collapses anyway is the point: the rejection loop is a selection pathology, not a capability deficit, and escaping it needs both field-based dispatch and the escalation arrow (Section 7.2). On the theoretical side, the one guarantee this coordination admits is that aligned local descent terminates, in productive ticks bounded by initial pressure rather than by the size of the state space.

Code Availability. Code is available at https://github.com/Govcraft/pressure-field-experiment. The repository and its CLI strategy key retain the project's historical name, *pressure field*; the system they implement is the pheromone-field strategy described in this paper.

Author Contributions. Sole author. All conceptualization, methodology, software implementation, formal analysis, investigation, and writing were performed by the author.

Declarations

Funding. No funding was received for conducting this study.

Competing Interests. The author has no relevant financial or non-financial interests to disclose.

Data Availability. All experimental data, analysis scripts, and raw trial results are available in the project repository: https://github.com/Govcraft/pressure-field-experiment.

Use of AI Tools. The author used Claude (Anthropic) to provide feedback on draft versions of this manuscript. All analytical decisions, experimental design, and final content were made by the author, who takes full responsibility for the work.

Appendix A Experimental Protocol

This appendix provides complete reproducibility information for all experiments.

A.1 Hardware and Software

Hardware: NVIDIA RTX PRO 5000 Laptop Graphics Processing Unit (GPU) (Blackwell, 24GB), Intel Core Ultra 9 275HX, 128GB RAM

Software:

- Rust 1.75+ (edition 2024)
- Ollama (local LLM inference server)
- Models: `qwen2.5:0.5b`, `qwen2.5:1.5b`, `qwen2.5:3b`

The environment runtime that ticks the logical clock, evaporates the pheromone stores, propagates activation, dispatches proposals, and serializes writes is implemented by a component named `KernelCoordinator`.

A.2 Model Configuration

Models are served via Ollama with a fixed system prompt:

```
You are a meeting room scheduler. Output schedules in the exact format
requested. No explanations, just the schedule.
```

A patch is the model's proposed schedule for one time block, parsed from a per-room listing; applying it rewrites that block only, and any meeting displaced by the rewrite returns to the unscheduled pool. For multi-model setups (model escalation), models share a single Ollama instance with automatic routing based on model name.

A.3 Sampling Diversity

Pheromone-field LLM calls sample temperature and top-p within one of three bands (Table A1); the band escalates with stagnation as described in Section 6.5. This diversity reduces premature convergence to local optima, though it cannot eliminate the legality stalls of Section 7.2.

A.4 Negative Pheromone Store

Alongside the positive example bank, a second pheromone store records harmful rejected patches as negative pheromones. On rejection of a harmful patch, the system deposits it on the artifact and injects it into later prompts for the same region. The injected

Band	Temperature	Top-p
Exploitation	0.15–0.35	0.80–0.90
Balanced	0.35–0.55	0.85–0.95
Exploration	0.55–0.85	0.90–0.98

Table A1 Sampling parameter ranges per band; temperature and top-p are sampled within range. Escalation proceeds Exploitation → Balanced → Exploration every 7 stalled ticks.

text is phrased as positive guidance ("TIP: schedule meetings in Room A") rather than a bare prohibition, restating what to try instead of what to avoid. This store evaporates on its own schedule, weight $\times 0.95$ per tick and evicted below 0.1, so stale avoid-tips lapse rather than permanently blocking a now-viable approach; up to three recent rejections per region are surfaced. The store carries no reinforcement on reuse, only deposit and evaporation.

A.5 Baseline Implementation Details

All strategies share identical LLM prompts, models, and parsing logic; only region selection and patch validation differ. The per-region prompt casts the model as a meeting room scheduler and supplies the block's time range, the rooms with their capacities, the current assignments for the block, the unscheduled meetings that could fit (with durations and attendee counts), and two constraints (no attendee double-booked, room capacity fits attendees), then asks for the schedule for that block.[2]

The strategies differ only in region selection and validation. Pheromone-field selects multiple regions concurrently by pressure gradient, validates all proposed patches, and applies only those that reduce pressure. Hierarchical always selects the single highest-pressure region and validates its one patch, applying it only if pressure reduces.

[2]Representative fields: `Time Block: [time range]; Rooms: Room A: capacity 10, Room B: capacity 8, ...; Current assignments: [...]; Unscheduled meetings that could fit: Meeting 5: 60min, 4 attendees; Meeting 12: 30min, 2 attendees; ...; Constraints: No attendee in multiple meetings at once; Room capacity must fit attendees; Output the schedule for this time block.`

Sequential round-robins through regions in fixed order and random selects uniformly; neither validates, so each applies any syntactically valid patch. Hierarchical processes one patch per tick by construction (rationale in Section 6): the controller observes global state, selects a region, delegates, validates, and applies within one tick.

The conversation baseline implements AutoGen-style dialogue with three roles, a *Coordinator* that selects a high-pressure region, a *Proposer* that suggests changes, and a *Validator* that critiques them. Each tick the Coordinator identifies a region (1 LLM call), then Proposer and Validator run up to 5 dialogue turns (2 calls per turn), and the patch applies if the Validator approves before the turn limit. This yields 4–12 LLM calls per tick for one region, versus pheromone-field's concurrent proposals across that tick's distinct high-pressure regions. It uses fixed sampling (T=0.3, top-p=0.9) for structured dialogue rather than pheromone-field's exploration bands. This is one instantiation inspired by AutoGen; actual deployments may use different role configurations, turn protocols, or termination conditions.

Fairness holds across baselines: all strategies use the identical model chain (qwen2.5:0.5b $\rightarrow$ 1.5b $\rightarrow$ 3b) with the same model-escalation trigger (21 stalled ticks), observe identical problem instances, receive the same 50-tick budget, and use identical output parsing. The conversation baseline's up-to-5 dialogue turns per region give it more refinement opportunity than pheromone-field's single-shot proposals.

A.6 Problem Generation and Seeding

Fair strategy comparison requires identical problem instances: each strategy must face the same scheduling challenge within a trial. We achieve this through deterministic seeding.

Each trial generates its problem from a seed:

$$\text{seed} = \text{trial} \times 1000 + \text{agent\_count}$$

Trial 5 with 2 agents yields seed 5002, producing identical meeting configurations whether evaluated with pheromone-field, conversation, or hierarchical coordination.

The seed governs all stochastic generation:

- Meeting durations (1–4 time slots)
- Attendee assignments (2–5 participants)
- Room preferences and capacity requirements
- Pre-scheduled vs. unassigned meeting distribution
- Time slot availability patterns

A.7 Experiment Commands

All experiments run with the default 50-tick budget; the CLI strategy key `pressure_field` is the project's historical name for the pheromone-field strategy. The main grid covers all five strategies, {1, 2, 4} agents, and {easy, medium, hard}; the ablation has 9 arms.

```
CHAIN=qwen2.5:0.5b,qwen2.5:1.5b,qwen2.5:3b
schedule-experiment --model-chain $CHAIN \
  grid --trials 30 --output results/main-grid.json
schedule-experiment --model-chain $CHAIN \
  ablation --trials 30 --agents 2 --difficulty medium \
  --output results/ablation.json
```

The scaling analysis (Section 6.4) and difficulty breakdown (Section 6.6) are slices of the main grid, not separate runs, so they inherit the grid's seeds and budgets.

A.8 Metrics Collected

Each experiment records:

- `solved`: Boolean indicating all meetings scheduled with zero overlaps
- `total_ticks`: Iterations to solve (or max if unsolved)
- `pressure_history`: Reported pressure at each tick ($1.0 \cdot$ unscheduled $+ 10.0 \cdot$ overlaps, the one authoritative metric)
- `band_escalation_events`: Sampling band changes (tick, from_band, to_band)
- `model_escalation_events`: Model tier changes (tick, from_model, to_model)
- `final_model`: Which model tier solved the schedule
- `token_usage`: Prompt and completion tokens consumed per trial

A.9 Replication Notes

Each configuration runs 30 independent trials with different random seeds to ensure reliability. Results report mean solve rates and tick counts across trials.

A.10 Measured Runtime

Table A2 reports wall-clock time for the two experiment campaigns on the hardware described in Appendix A.1. The main grid (1350 trials) and the nine-arm ablation (270 trials) together took about 100 hours of GPU time. Wall time is set by the number of sequential model calls, not by per-call GPU throughput: an unsolved trial runs the full fifty-tick budget, and each tick issues up to about twenty calls, one per activated region. A faster GPU shortens each call but not their count, so aggregate time scales with the volume of inference. Per-trial cost concentrates on the harder instances, where pressure stagnation escalates proposals to the larger models in the chain, lengthening individual calls without reducing their number.

Appendix B Pressure Alignment Verification

This appendix verifies that the meeting scheduling domain satisfies the alignment condition required for convergence (Lemma 5.1).

Experiment	Configurations	Trials each	Wall time
Main Grid	45	30	58.9 h
Ablation	9	30	41.6 h

Table A2 Measured wall time on the NVIDIA RTX PRO 5000 GPU (24GB). The scaling and difficulty analyses are slices of the main grid.

B.1 Per-Region Pressure Components

The per-region pressure function includes three components:

- **gap_ratio**: Fraction of empty slots in this time block. *Strictly local*:depends only on meetings scheduled within this region.
- **overlap_count**: Number of attendee double-bookings within this time block. *Strictly local*:the overlap sensor counts attendees with multiple meetings in *this block only*, not across blocks.
- **utilization_variance**: Variance in room utilization within this time block. *Strictly local*:measures balance across rooms for meetings in this region.

The `unscheduled_count` component is added to *total* pressure only, not to per-region pressure. This design choice ensures separability.

B.2 Coupling Analysis

For the alignment condition, we need $|P_j(s') - P_j(s)| \leq \epsilon$ for all $j \neq i$ when modifying region i.

Result: $\epsilon = 0$ (per-region pressure is separable).

Rationale: All three per-region components depend only on the region's own content:

1. Modifying region i's schedule changes which meetings occupy region i's slots.

2. This affects region i's gap_ratio, overlap_count, and utilization_variance.
3. It has *zero effect* on any other region j's pressure, since j's components depend only on meetings within j's time slots.

Note on cross-region movement: A patch rewrites exactly one block, so no single action modifies two regions. A meeting crosses blocks in two steps through the unscheduled pool: one patch displaces it (changing only the evicting region's components and the global unscheduled term, which belongs to no region), and a later patch on another region places it. Each step changes only its own region's per-region pressure, so $\epsilon = 0$ holds for the actual action set. The acceptance gate closes the remaining gap: because it evaluates the total metric P, including the unscheduled term, a patch that trades an overlap for pool meetings is accepted exactly when the total strictly decreases, so alignment holds by construction as well as by separability.

B.3 Empirical Validation

Analysis of the pheromone-field trials confirms separability:

- Total tick-to-tick transitions analyzed: 9,856
- Transitions with pressure improvement: 1,102
- Transitions with pressure degradation: 2
- Transitions with no change: 8,752

The near-total absence of pressure degradation (2 of 9,856 transitions, 0.02%) is consistent with separable pressure: each accepted patch reduces local pressure, which directly reduces total pressure without adverse effects on other regions, because patches that would increase local pressure are rejected by validation.

Mean improvement magnitude is 2.59 pressure units against a mean initial pressure of 12.8 units. This exceeds any plausible coupling bound, confirming that local improvements translate to global improvements.

Verification of $\delta_{\min} > 0$: Lemma 5.1 requires $\delta_{\min} > (n-1)\epsilon$. With $\epsilon = 0$ (separable pressure), this reduces to $\delta_{\min} > 0$. Empirically, the minimum observed pressure reduction across accepted patches is 1.0 pressure units, confirming $\delta_{\min} > 0$. The convergence guarantee therefore applies to this domain.

Appendix C Proof of the Convergence Lemma

This appendix proves Lemma 5.1 (Termination of Aligned Descent) from Section 5.

Proof Let $P^t = P(s^t)$ denote total pressure at tick t. Under ϵ-bounded coupling, when a patch reduces a region's own pressure by δ_i, the total changes by

$$P^{t+1} - P^t = -\delta_i + \sum_{j \neq i} (P_j(s^{t+1}) - P_j(s^t)),$$

and bounded coupling gives $|P_j(s^{t+1}) - P_j(s^t)| \leq \epsilon$ for each of the $n-1$ other regions, so

$$P^{t+1} - P^t \leq -\delta_i + (n-1)\epsilon.$$

If $\delta_i \geq \delta_{\min}$ and $\delta_{\min} > (n-1)\epsilon$, then each productive tick lowers the total by at least $\delta_{\min} - (n-1)\epsilon > 0$. Since $P(s) \geq 0$ for all states, a non-negative quantity decreasing by this fixed positive amount per productive tick reaches a state where no region exceeds τ_{act} within

$$T \leq \frac{P_0}{\delta_{\min} - (n-1)\epsilon}$$

ticks; at that point no actor is dispatched. □

References

Agashe S, Fan Y, et al (2025) LLM-Coordination: Evaluating and analyzing multi-agent coordination abilities in large language models. In: Findings of the 2025 Conference

of the North American Chapter of the Association for Computational Linguistics (NAACL), https://doi.org/10.18653/v1/2025.findings-naacl.448

Aina OA, et al (2025) Deep reinforcement learning for multi-agent coordination (S-MADRL). Artificial Life and Robotics https://doi.org/10.1007/s10015-025-01089-z

Amaral CJ, Hübner JF, Cranefield S (2023) Generating and choosing organisations for multi-agent systems. Autonomous Agents and Multi-Agent Systems 37(2):41. https://doi.org/10.1007/s10458-023-09623-8

Babaoglu O, Canright G, Deutsch A, et al (2006) Design patterns from biology for distributed computing. ACM Transactions on Autonomous and Adaptive Systems 1(1):26–66. https://doi.org/10.1145/1152934.1152937

Bonabeau E, Henaux F, Guérin S, et al (1998) Routing in telecommunications networks with ant-like agents. In: Intelligent Agents for Telecommunication Applications (IATA 1998), LNCS, vol 1437. Springer, pp 60–71, https://doi.org/10.1007/BFb0053944

Bratman ME (1987) Intention, Plans, and Practical Reason. Harvard University Press, Cambridge, MA

Brueckner SA (2000) Return from the ant: Synthetic ecosystems for manufacturing control. PhD thesis, Humboldt University Berlin, Department of Computer Science

Brueckner SA, Parunak HVD, Hilscher R (2009) Swarming polyagents executing hierarchical task networks. In: Third IEEE International Conference on Self-Adaptive and Self-Organizing Systems (SASO 2009). IEEE, pp 51–60, https://doi.org/10.1109/SASO.2009.21

Buscemi A, et al (2025) When numbers start talking: Implicit numerical coordination among LLM-based agents. arXiv preprint arXiv:260103846 https://doi.org/10.48550/arXiv.2601.03846

Camazine S, Deneubourg JL, Franks NR, et al (2001) Self-Organization in Biological Systems. Princeton Studies in Complexity, Princeton University Press, Princeton, NJ

Claes R, Holvoet T, Weyns D (2011) A decentralized approach for anticipatory vehicle routing using delegate multiagent systems. IEEE Transactions on Intelligent Transportation Systems 12(2):364–373. https://doi.org/10.1109/TITS.2011.2105867

Cohen PR, Levesque HJ (1991) Teamwork. Noûs 25(4):487–512. https://doi.org/10.2307/2216075, special Issue on Cognitive Science and Artificial Intelligence

De Wolf T, Holvoet T (2005) Towards a methodology for engineering self-organising emergent systems. In: Self-Organization and Autonomic Informatics (I). IOS Press

Dorigo M, Gambardella LM (1997) Ant colony system: A cooperative learning approach to the traveling salesman problem. IEEE Transactions on Evolutionary Computation 1(1):53–66. https://doi.org/10.1109/4235.585892

Dorigo M, Stützle T (2004) Ant Colony Optimization. MIT Press, Cambridge, MA

Dorigo M, Maniezzo V, Colorni A (1996) Ant system: Optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics – Part B 26(1):29–41. https://doi.org/10.1109/3477.484436

Du Y, Li S, Torralba A, et al (2024) Improving factuality and reasoning in language models through multiagent debate. In: Proceedings of the Forty-First International Conference on Machine Learning (ICML), https://doi.org/10.48550/arXiv.2305.14325

Grassé PP (1959) La reconstruction du nid et les coordinations interindividuelles chez Bellicositermes natalensis et Cubitermes sp. la théorie de la stigmergie. Insectes Sociaux 6:41–80. https://doi.org/10.1007/BF02223791

Guo T, Chen X, Wang Y, et al (2024) Large language model based multi-agents: A survey of progress and challenges. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI), Survey Track, pp 8048–8057, https://doi.org/10.24963/ijcai.2024/890

Hadeli, Valckenaers P, Kollingbaum M, et al (2004) Multi-agent coordination and control using stigmergy. Computers in Industry 53(1):75–96. https://doi.org/10.1016/S0166-3615(03)00123-4

Han B, et al (2025) Exploring advanced LLM multi-agent systems based on blackboard architecture (LbMAS). arXiv preprint arXiv:250701701 https://doi.org/10.48550/arXiv.2507.01701

Hong S, Zhuge M, Chen J, et al (2024) MetaGPT: Meta programming for a multi-agent collaborative framework. In: Proceedings of the Twelfth International Conference on Learning Representations (ICLR), https://doi.org/10.48550/arXiv.2308.00352

Jimenez Romero C, et al (2025) Multi-agent systems powered by large language models: Applications in swarm intelligence. Frontiers in Artificial Intelligence https://doi.org/10.3389/frai.2025.1593017

Kim M, et al (2025) Towards a science of scaling agent systems. arXiv preprint arXiv:251208296 https://doi.org/10.48550/arXiv.2512.08296

Li G, Hammoud HAAK, Itani H, et al (2023) CAMEL: Communicative agents for "mind" exploration of large language model society. In: Advances in Neural Information Processing Systems (NeurIPS), https://doi.org/10.48550/arXiv.2303.17760

Mamei M, Zambonelli F (2009) Programming pervasive and mobile computing applications: The TOTA approach. ACM Transactions on Software Engineering and Methodology 18(4):15:1–15:56. https://doi.org/10.1145/1538942.1538945

Monderer D, Shapley LS (1996) Potential games. Games and Economic Behavior 14:124–143. https://doi.org/10.1006/game.1996.0044

Park JS, O'Brien JC, Cai CJ, et al (2023) Generative agents: Interactive simulacra of human behavior. In: Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST), https://doi.org/10.1145/3586183.3606763

Parunak HVD (1997) "go to the ant": Engineering principles from natural multi-agent systems. Annals of Operations Research 75:69–101. https://doi.org/10.1023/A:1018980001403

Parunak HVD (2006) A survey of environments and mechanisms for human-human stigmergy. In: Weyns D, Parunak HVD, Michel F (eds) Environments for Multi-Agent Systems II (E4MAS 2005), Selected Revised and Invited Papers, Lecture Notes in Computer Science, vol 3830. Springer, pp 163–186, https://doi.org/10.1007/11678809_10

Parunak HVD (2011) Between agents and mean fields. In: Multi-Agent-Based Simulation XII (MABS 2011, at AAMAS 2011). Springer, Taipei, Taiwan, LNAI, pp 106–117

Parunak HVD (2025) Insects and agents: Extending the metaphor. In: Multi-Agent-Based Simulation XXVI (MABS 2025, at AAMAS 2025). Springer, LNCS

Qian C, Liu W, Liu H, et al (2024) ChatDev: Communicative agents for software development. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp 15174–15186, https://doi.org/10.18653/v1/2024.acl-long.810

Qian C, Zuo J, Chen W, et al (2025) MacNet: Scaling large language model-based multi-agent collaboration. In: Proceedings of the Thirteenth International Conference

on Learning Representations (ICLR), https://doi.org/10.48550/arXiv.2406.07155

Rao AS, Georgeff MP (1995) BDI agents: From theory to practice. Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95) pp 312–319

Ren S, Hu Z, Hu S, et al (2024) Emergence of social norms in generative agent societies: Principles and architecture. In: Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI), HAI Track, https://doi.org/10.24963/ijcai.2024/874

Riedl MO (2025) Emergent coordination in multi-agent language models. arXiv preprint arXiv:251005174 https://doi.org/10.48550/arXiv.2510.05174

Sauter JA, Matthews R, Parunak HVD, et al (2005) Performance of digital pheromones for swarming vehicle control. In: Proceedings of the Fourth International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2005). ACM, pp 903–910, https://doi.org/10.1145/1082473.1082610

Savarimuthu BTR, Ranathunga S, Cranefield S (2024) Harnessing the power of LLMs for normative reasoning in MASs. In: Coordination, Organizations, Institutions, Norms, and Ethics for Governance of Multi-Agent Systems XVII (COINE 2024, at AAMAS 2024), LNCS, vol 15398. Springer, p 132–145, https://doi.org/10.1007/978-3-031-82039-7_9

Serugendo GDM, Gleizes MP, Karageorgos A (2005) Self-organisation in multi-agent systems. The Knowledge Engineering Review 20(2):165–189. https://doi.org/10.1017/S0269888905000494

Valckenaers P, Van Brussel H (2015) Design for the Unexpected: From Holonic Manufacturing Systems towards a Humane Mechatronics Society. Butterworth-Heinemann

Valmeekam K, Marquez M, Sreedharan S, et al (2023) On the planning abilities of large language models—a critical investigation. In: Advances in Neural Information Processing Systems (NeurIPS), https://doi.org/10.48550/arXiv.2305.15771

Weyns D, Michel F, Parunak HVD, et al (eds) (2015) Agent Environments for Multi-Agent Systems — A Research Roadmap. Springer, Berlin

Wu Q, Bansal G, Zhang J, et al (2023) AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:230808155 https://doi.org/10.48550/arXiv.2308.08155

Xu Y, Deng C, Chen X, et al (2022) Stigmergic independent reinforcement learning for multiagent collaboration. IEEE Transactions on Neural Networks and Learning Systems 33(9):4285–4299. https://doi.org/10.1109/TNNLS.2021.3056418

Zhang G, et al (2025) Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. In: Proceedings of the Thirteenth International Conference on Learning Representations (ICLR), https://doi.org/10.48550/arXiv.2410.02506